

BÀI 5. QUẢN LÝ TRẠNG THÁI VỚI SESSION

Tuần trước, chúng ta đã có một giao diện đẹp mắt. Nhưng nếu bạn thử thêm một sản phẩm vào giỏ hàng (giả sử đã có nút thêm vào giỏ hàng), rồi chuyển sang trang khác, giỏ hàng của bạn sẽ trống rỗng. Tại sao? Vì web vốn dĩ "não cá vàng". Tuần này, chúng ta sẽ cấp cho ứng dụng của mình một "bộ nhớ tạm" mạnh mẽ gọi là **Session** để nó có thể "nhớ" được giỏ hàng của từng người dùng.

MỤC TIÊU BÀI HỌC: Sau khi hoàn thành tuần học này, bạn sẽ có khả năng:

1. **Giải thích được bản chất stateless** của giao thức HTTP và tại sao nó là một thách thức.
2. **Trình bày rành mạch cơ chế hoạt động** của Session, từ Session ID, cookie cho đến kho lưu trữ phía server.
3. **Tự tay cấu hình** và sử dụng Session trong một ứng dụng ASP.NET Core.
4. **Viết và sử dụng Extension Methods** để lưu trữ các đối tượng phức tạp vào Session một cách thanh lịch và chuyên nghiệp.

NỘI DUNG LÝ THUYẾT

5.1. HTTP Stateless và Các Kỹ thuật Quản lý Trạng thái

5.1.1. HTTP là Stateless: Giao thức "hay quên"

Hãy tưởng tượng bạn nói chuyện với một người bị mất trí nhớ ngắn hạn. Mỗi câu bạn nói, họ đều coi như là lần đầu tiên gặp bạn.

- **Bạn:** "Chào bạn, tôi muốn mua quả bóng đá."
- **Người bán:** "Vâng, của bạn đây."
- **Bạn (ở một quầy khác):** "Giờ tôi muốn thanh toán."
- **Người bán:** "Thanh toán cái gì? Tôi có biết bạn là ai đâu?"

HTTP (HyperText Transfer Protocol) – giao thức nền tảng của web – hoạt động y hệt như vậy. Mỗi request từ trình duyệt của bạn đến server là một sự kiện **hoàn toàn độc lập**. Server không có ký ức gì về các request bạn đã gửi trước đó. Đây được gọi là **stateless (vô trạng thái)**.

Đây là một thiết kế có chủ đích để làm cho web đơn giản và có khả năng mở rộng, nhưng nó tạo ra một vấn đề lớn: làm thế nào để duy trì các thông tin như "người dùng đã đăng nhập" hay "giỏ hàng có những gì"?

5.1.2. Các phương pháp duy trì trạng thái: "Giấy nhớ" cho người hay quên

Để giúp server "nhớ", chúng ta cần một cách để client "nhắc nhở" server về các thông tin cũ trong mỗi request mới.

Phương pháp	Cách hoạt động	Ưu điểm	Nhược điểm
Query Strings	Viết thông tin trực tiếp lên URL. site.com/cart?itemId=5	Đơn giản, dễ chia sẻ.	Lộ thông tin, giới hạn độ dài, xấu.
Hidden Fields	Giấu thông tin trong các thẻ <code><input type="hidden"></code> của form.	Hữu ích để truyền dữ liệu giữa các bước của một form.	Chỉ hoạt động với form POST, không duy trì qua các trang khác.
Cookies	Server gửi một mẫu tin nhỏ (cookie) và trình duyệt sẽ tự động gửi lại mẫu tin đó trong mọi request tiếp theo đến cùng server.	Tự động, bền bỉ (có thể lưu lâu dài).	Giới hạn kích thước (khoảng 4KB), không an toàn cho dữ liệu nhạy cảm.
Session State	Cách làm thông minh nhất: Server chỉ gửi cho trình duyệt một cái "vé gửi đồ" (là một cookie chứa Session ID). Dữ liệu thật (giỏ hàng) được cất trong một "tủ đồ" an toàn trên server.	Bảo mật (dữ liệu thật không rời khỏi server), không giới hạn kích thước.	Tốn tài nguyên server (bộ nhớ).

Trong bài toán giỏ hàng, **Session State** là lựa chọn tối ưu vì nó vừa bảo mật, vừa linh hoạt.

5.2. Session trong ASP.NET Core - "Phòng gửi đồ" của Server

5.2.1. Session hoạt động như thế nào?

Hãy tưởng tượng quy trình này như bạn đi siêu thị và dùng tủ gửi đồ (locker):

1. Request đầu tiên (Bạn đến siêu thị lần đầu):

- Trình duyệt của bạn gửi một request đến server.
- Middleware Session của ASP.NET Core thấy bạn là khách mới, chưa có "vé".
- Nó sẽ:
 - Tạo một **Session ID** duy nhất (ví dụ: ABC-123-XYZ).
 - Mở một "ngăn tủ" trống trên bộ nhớ của server.
 - Gửi một **cookie** chứa Session ID (ABC-123-XYZ) về cho trình duyệt của bạn. Cái cookie này chính là **chìa khóa tủ**.

2. Lưu vào Session (Bạn gửi đồ vào tủ):

- Trong code, khi bạn gọi `HttpContext.Session.SetString("cart", "...")`, bạn đang nói: "Hãy cất món đồ này vào ngăn tủ có chìa khóa ABC-123-XYZ".

3. Các Request tiếp theo (Bạn quay lại siêu thị):

- Khi bạn truy cập một trang khác, trình duyệt sẽ tự động đính kèm **cookie "chìa khóa tủ"** vào request.
- Middleware Session trên server nhận được request, nhìn vào chìa khóa và nói: "À, đây là khách ABC-123-XYZ". Nó sẽ tìm đúng ngăn tủ của bạn và mở nó ra, sẵn sàng để bạn lấy đồ hoặc cất thêm đồ.

5.2.2. Cấu hình Session trong Program.cs

Để "phòng gửi đồ" này hoạt động, bạn phải thực hiện 3 bước cài đặt trong Program.cs.

```
var builder = WebApplication.CreateBuilder(args);

// ... các dịch vụ khác

// BƯỚC 1: Cung cấp kho lưu trữ cho Session (Xây dựng các dãy tủ locker)
// Session cần một nơi để cất dữ liệu. AddDistributedMemoryCache() cung cấp
```

Bài 5 – Quản lý trạng thái với session

```
// một kho lưu trữ ngay trong bộ nhớ của server. Đây là lựa chọn đơn giản nhất.
builder.Services.AddDistributedMemoryCache();

// BƯỚC 2: Đăng ký dịch vụ Session (Thuê nhân viên trông tủ)
// Dòng này đăng ký các dịch vụ cần thiết để quản lý việc tạo Session ID,
// đọc/ghi cookie, và truy cập vào kho lưu trữ.
builder.Services.AddSession(options =>
{
    // Bạn có thể cấu hình thêm, ví dụ: thời gian hết hạn của session
    options.IdleTimeout = TimeSpan.FromMinutes(30);
});

var app = builder.Build();

// ... các middleware khác như UseStaticFiles()

// BƯỚC 3: Kích hoạt Middleware Session (Mở cửa phòng gửi đồ)
// Middleware là một "trạm kiểm soát" cho mỗi request. UseSession() sẽ
// tự động kiểm tra cookie "chìa khóa" trên mỗi request đến và
// thiết lập HttpContext.Session để bạn có thể sử dụng trong Controller.
// QUAN TRỌNG: Nó phải được gọi trước UseRouting() và UseEndpoints().
app.UseSession();

app.UseRouting();
// ...
```

5.2.3. Các thao tác cơ bản

Sau khi đã cấu hình, bạn có thể truy cập Session trong Controller thông qua đối tượng `HttpContext.Session`:

```
// Lưu một chuỗi
HttpContext.Session.SetString("UserName", "Alice");

// Đọc một chuỗi
string? userName = HttpContext.Session.GetString("UserName");

// Lưu một số nguyên
HttpContext.Session.SetInt32("UserAge", 30);
```

```
// Đọc một số nguyên  
int? userAge = HttpContext.Session.GetInt32("UserAge");
```

5.3. Extension Methods và Lưu trữ Đối tượng trong Session

5.3.1. Vấn đề: Tủ locker chỉ nhận được "giấy nhớ"

Như bạn thấy ở trên, các phương thức Set/Get mặc định chỉ làm việc với các kiểu dữ liệu đơn giản (nguyên thủy) như string, int. Nhưng giỏ hàng của chúng ta là một đối tượng Cart phức tạp, chứa một danh sách các CartLine. Làm sao để cất cả một "cái xe đầy" vào một ngăn tủ chỉ nhận "giấy nhớ"?

5.3.2. Giải pháp: "Chụp ảnh" đối tượng bằng JSON

Serialization là quá trình chuyển đổi một đối tượng trong bộ nhớ (như object Cart) thành một định dạng có thể lưu trữ hoặc truyền đi được (như một chuỗi văn bản). **JSON (JavaScript Object Notation)** là định dạng phổ biến nhất cho việc này.

– Quy trình:

1. **Serialize (Chụp ảnh):** Trước khi lưu, chúng ta "chụp ảnh" đối tượng Cart và biến nó thành một chuỗi JSON.
2. **Lưu trữ:** Lưu chuỗi JSON này vào Session bằng SetString().
3. **Đọc ra:** Lấy chuỗi JSON từ Session bằng GetString().
4. **Deserialize (Tái tạo):** Dùng chuỗi JSON để "tái tạo" lại đối tượng Cart như ban đầu.

5.3.3. Sự bất tiện và giải pháp Extension Methods

Nếu làm theo cách trên, code trong Controller của bạn sẽ trông như thế này:

```
// Lưu giỏ hàng  
var cartAsJson = JsonSerializer.Serialize(myCart);  
HttpContext.Session.SetString("Cart", cartAsJson);  
  
// Lấy giỏ hàng  
var storedJson = HttpContext.Session.GetString("Cart");  
var myCart = JsonSerializer.Deserialize<Cart>(storedJson);
```

Việc lặp lại code này ở nhiều nơi rất nhàm chán và dễ gây lỗi. Chúng ta cần một cách làm tốt hơn:

Extension Methods (Phương thức mở rộng): "Dạy" cho Session những kỹ năng mới

- Extension Method cho phép bạn thêm các phương thức mới vào một kiểu dữ liệu đã có mà không cần sửa mã nguồn gốc của nó.
- Chúng ta sẽ "dạy" cho ISession hai kỹ năng mới là SetJson() và GetJson().

HƯỚNG DẪN THỰC HÀNH

Bước 1: File SessionExtensions.cs (Công cụ chuyển đổi JSON)

Vì Session mặc định không lưu được Object phức tạp, chúng ta cần một công cụ chuyển đổi Object thành chuỗi JSON.

- **Thao tác:** Tạo thư mục Infrastructure -> Tạo file SessionExtensions.cs.
- Code:

```
// Nạp thư viện hỗ trợ xử lý JSON của .NET
using System.Text.Json;

// Định nghĩa không gian tên (namespace) để các file khác có thể gọi đến
namespace SportsStore.WebUI.Infrastructure;

// Bắt buộc phải là class 'static' (tĩnh) để chứa các Extension Method
public static class SessionExtensions
{
    // Cú pháp 'this ISession session' giúp hàm SetJson gắn chặt vào mọi
    // biến kiểu ISession.
    // 'key' là tên ngăn kéo trong Session, 'value' là món đồ (object) bạn
    // muốn cất.
    public static void SetJson(this ISession session, string key, object
    value)
    {
        // Biến 'value' (ví dụ: giỏ hàng) thành một chuỗi văn bản định dạng
        JSON
        // Sau đó lưu chuỗi đó vào Session thông qua hàm SetString mặc
        // định.
        session.SetString(key, JsonSerializer.Serialize(value));
    }

    // '<T>' báo hiệu đây là hàm Generic (chấp nhận mọi kiểu dữ liệu).
    // Hàm này trả về kiểu T? (có thể null nếu không tìm thấy dữ liệu).
    public static T? GetJson<T>(this ISession session, string key)
```

```
{
    // Lấy chuỗi JSON thô từ Session dựa vào 'key'
    var sessionData = session.GetString(key);

    // Toán tử 3 ngôi: Nếu chuỗi JSON bị rỗng (chưa từng lưu gì vào key
    này)
    return sessionData == null
        ? default(T) // Trả về giá trị mặc định của T (thường là null
        đối với Object)
        // Nếu có chữ, dịch ngược chuỗi JSON đó thành Object kiểu T ban
        đầu
        : JsonSerializer.Deserialize<T>(sessionData);
}
```

Bước 2: File Cart.cs (Cấu trúc dữ liệu)

Định nghĩa hình dáng của một dòng sản phẩm và toàn bộ giỏ hàng.

- **Thao tác:** Trong thư mục Models, tạo file Cart.cs.
- **Code:**

```
namespace SportsStore.WebUI.Models;

// Lớp đại diện cho MỘT DÒNG SẢN PHẨM trong giỏ hàng
public class CartLine
{
    // Mã tự tăng của dòng (không bắt buộc dùng tới)
    public int CartLineId { get; set; }
    // Chứa toàn bộ thông tin sản phẩm (Tên, Giá...)
    public Product Product { get; set; } = new();
    // Số lượng khách đặt mua cho sản phẩm này
    public int Quantity { get; set; }
}

// Lớp đại diện cho TOÀN BỘ GIỎ HÀNG
public class Cart
{
    // Tạo một danh sách rỗng chứa các dòng sản phẩm
    public List<CartLine> Lines { get; set; } = new List<CartLine>();

    // Hàm thêm sản phẩm vào giỏ
}
```

```
public virtual void AddItem(Product product, int quantity)
{
    // Quét trong giỏ xem có sản phẩm nào trùng ID với sản phẩm đang
    muốn thêm không
    CartLine? line = Lines.FirstOrDefault(p => p.Product.ProductID ==
product.ProductID);

    // Nếu 'line' là null nghĩa là sản phẩm này chưa từng có trong giỏ
    if (line == null)
    {
        // Tạo một dòng mới và nhét vào danh sách Lines
        Lines.Add(new CartLine { Product = product, Quantity = quantity
});
    }
    else // Nếu sản phẩm đã nằm trong giỏ rồi
    {
        // Chỉ cần cộng dồn số lượng mới vào số lượng cũ
        line.Quantity += quantity;
    }
}

// Hàm xóa hoàn toàn một sản phẩm khỏi giỏ dựa vào ID
public virtual void RemoveLine(Product product) =>
    Lines.RemoveAll(l => l.Product.ProductID == product.ProductID);

// Hàm tính tổng tiền: Duyệt qua từng dòng, lấy (Giá * Số lượng), rồi
cộng tổng (Sum) lại
public decimal ComputeTotalValue() =>
    Lines.Sum(e => e.Product.Price * e.Quantity);

// Hàm xóa sạch giỏ hàng (thường gọi sau khi khách thanh toán xong)
public virtual void Clear() => Lines.Clear();
}
```

Bước 3: File Program.cs (Cấu hình hệ thống)

Báo cho ứng dụng biết bạn sẽ sử dụng bộ nhớ tạm để lưu giỏ hàng.

- **Thao tác:** Mở Program.cs. Thêm các dòng cấu hình vào đúng vị trí theo thứ tự.
- **Code:**

```
// Đăng ký bộ nhớ đệm dạng Ram (Session cần cái này để hoạt động)
```



```
builder.Services.AddDistributedMemoryCache();

// Đăng ký dịch vụ Session vào hệ thống
builder.Services.AddSession(options => {
    // Nếu khách không thao tác gì trên web trong 30 phút, giỏ hàng sẽ bị xóa
    options.IdleTimeout = TimeSpan.FromMinutes(30);
    // Bảo mật: Không cho phép JavaScript ở trình duyệt đọc được Cookie chứa ID Session
    options.Cookie.HttpOnly = true;
    // Đánh dấu Cookie này là thiết yếu, không bị chặn bởi các quy định GDPR (luật bảo vệ dữ liệu)
    options.Cookie.IsEssential = true;
});

var app = builder.Build();

// ...

// Yêu cầu ứng dụng thực sự BẬT tính năng Session lên (nếu thiếu dòng này, app sẽ báo lỗi)
// Bắt buộc phải nằm TRƯỚC dòng app.MapControllerRoute(...)
app.UseSession();
```

4. File CartController.cs (Xử lý Logic)

Đây là trái tim của hệ thống. [HttpPost] để chống lỗi hỏng bảo mật và xử lý returnUrl để người dùng không bị "đá văng" sang trang khác một cách vô lý.

- **Thao tác:** Trong Controllers, tạo CartController.cs.
- **Code:**

```
using Microsoft.AspNetCore.Mvc;
using SportsStore.WebUI.Models;
using SportsStore.WebUI.Infrastructure;

namespace SportsStore.WebUI.Controllers;

public class CartController : Controller
{
    // Dữ liệu giả lập thay cho Database thực tế
    private readonly List<Product> _mockDatabase = new List<Product>
```

```
{
    new Product { ProductID = 1, Name = "Giày chạy bộ", Price = 100 },
    new Product { ProductID = 2, Name = "Vợt cầu lông", Price = 50 }
};

// Hàm dùng chung: Mọi giỏ hàng từ Session ra
private Cart GetCart()
{
    // Gọi hàm GetJson (đã viết ở bước 1). Nếu Session "Cart" rỗng, thì
    // tạo mới 'new Cart()'
    return HttpContext.Session.GetJson<Cart>("Cart") ?? new Cart();
}

// Hành động (Action) hiển thị trang Giỏ hàng. 'returnUrl' là link
// trang mà khách vừa đứng.
public IActionResult Index(string returnUrl)
{
    // Nhét returnUrl vào ViewBag để View có thể lấy ra làm link cho
    // nút "Quay lại"
    ViewBag.ReturnUrl = returnUrl;

    // Lấy giỏ hàng và gửi xuống cho View hiển thị
    return View(GetCart());
}

// Đánh dấu [HttpPost] để bảo mật, chỉ cho phép form submit, chặn việc
// gõ URL trực tiếp
[HttpPost]
public IActionResult AddToCart(int productId, string returnUrl)
{
    // Tìm xem productId khách gửi lên có tồn tại trong Database không
    Product? product = _mockDatabase.FirstOrDefault(p => p.ProductID ==
productId);

    if (product != null) // Nếu tìm thấy hàng
    {
        Cart cart = GetCart(); // 1. Lấy giỏ hàng hiện tại
        cart.AddItem(product, 1); // 2. Thêm 1 sản phẩm vào giỏ
        HttpContext.Session.SetJson("Cart", cart); // 3. Lưu giỏ
        // hàng mới vào lại Session
    }
}
```

```
// Nếu có returnUrl (khách bấm thêm từ trang danh sách), đã khách
về lại trang danh sách đó
if (!string.IsNullOrEmpty(returnUrl))
{
    return LocalRedirect(returnUrl);
}

// Nếu không có returnUrl, đã khách sang trang xem chi tiết Giỏ
hàng (Action Index ở trên)
return RedirectToAction("Index");
}

[HttpPost]
public IActionResult RemoveFromCart(int productId, string returnUrl)
{
    Product? product = _mockDatabase.FirstOrDefault(p => p.ProductID ==
productId);
    if (product != null)
    {
        Cart cart = GetCart();
        cart.RemoveLine(product); // Gọi hàm xóa khỏi giỏ
        HttpContext.Session.SetJson("Cart", cart); // Lưu cập nhật vào
Session
    }

    // Xóa xong thì load lại trang Giỏ hàng, đồng thời nhớ mang theo
returnUrl cũ (nếu có)
    return RedirectToAction("Index", new { returnUrl });
}
}
```

Bước 5: File Index.cshtml (Giao diện hiển thị)

Hiển thị bảng tóm tắt sản phẩm và tổng tiền, tích hợp link điều hướng thông minh.

- **Thao tác:** Tạo thư mục Views/Cart -> Tạo file Index.cshtml.
- **Code:**

```
@* Khai báo kiểu dữ liệu mà Controller gửi xuống chính là class Cart *@
@model SportsStore.WebUI.Models.Cart

@{
```

```
// Cài đặt tiêu đề hiển thị trên Tab của trình duyệt
ViewData["Title"] = "Giỏ hàng của bạn";
}

<h2 class="mb-4">Giỏ hàng của bạn</h2>

<table class="table table-bordered table-striped">
  <thead class="table-dark">
    <tr>
      <th>Số lượng</th>
      <th>Sản phẩm</th>
      <th class="text-end">Đơn giá</th>
      <th class="text-end">Thành tiền</th>
      <th class="text-center">Hành động</th>
    </tr>
  </thead>
  <tbody>
    *@ Bắt đầu viết C#: Nếu danh sách Lines trong giỏ hàng rỗng (bằng
0) *@
    @if (Model.Lines.Count == 0)
    {
      <tr><td colspan="5" class="text-center text-muted">Giỏ hàng
trống</td></tr>
    }

    *@ Vòng lặp C#: Quét qua từng dòng sản phẩm trong giỏ hàng *@
    @foreach (var line in Model.Lines)
    {
      <tr>
        <td class="text-center">@line.Quantity</td>

        <td>@line.Product.Name</td>

        <td class="text-end">@line.Product.Price.ToString("C")</td>

        <td class="text-end">@((line.Quantity *
line.Product.Price).ToString("C"))</td>

        <td class="text-center">
          <form asp-action="RemoveFromCart" method="post">
            <input type="hidden" name="productId">
```

```
value="@line.Product.ProductID" />
                <input type="hidden" name="returnUrl"
value="@ViewBag.ReturnUrl" />

                <button type="submit" class="btn btn-sm btn-
danger">Xóa</button>
            </form>
        </td>
    </tr>
}
</tbody>
<tfoot>
    <tr>
        <td colspan="3" class="text-end fw-bold">Tổng cộng:</td>
        <td class="text-end fw-bold text-
danger">@Model.ComputeTotalValue().ToString("C")</td>
        <td></td>
    </tr>
</tfoot>
</table>

<div class="text-center mt-3">
    <a class="btn btn-secondary" href="@ (ViewBag.ReturnUrl ?? "/" )">Tiếp
tục mua sắm</a>

    <a class="btn btn-success" asp-controller="Order" asp-
action="Checkout">Thanh toán ngay</a>
</div>
```

Bước 6: Tạo nút "Thêm vào giỏ" ở trang chủ để test

Mở file Views/Home/Index.cshtml (hoặc bất kỳ trang nào bạn đang dùng làm trang chính) và dán đoạn mã này vào để tạo ra 2 sản phẩm giống hệt trong Database giả lập của chúng ta:

```
@{
    ViewData["Title"] = "Trang chủ";
}

<div class="container mt-5">
    <h2>Danh sách sản phẩm</h2>
    <div class="row mt-4">
```

```
<div class="col-md-4">
    <div class="card p-3 shadow-sm">
        <h4>Giày chạy bộ</h4>
        <p class="text-danger fw-bold">Giá: $100.00</p>

        <form asp-controller="Cart" asp-action="AddToCart"
method="post">
            <input type="hidden" name="productId" value="1" />
            <input type="hidden" name="returnUrl"
value="@Context.Request.Path" />
            <button type="submit" class="btn btn-primary">Thêm vào
giỏ</button>
        </form>
    </div>
</div>

<div class="col-md-4">
    <div class="card p-3 shadow-sm">
        <h4>Vợt cầu lông</h4>
        <p class="text-danger fw-bold">Giá: $50.00</p>

        <form asp-controller="Cart" asp-action="AddToCart"
method="post">
            <input type="hidden" name="productId" value="2" />
            <input type="hidden" name="returnUrl"
value="@Context.Request.Path" />
            <button type="submit" class="btn btn-primary">Thêm vào
giỏ</button>
        </form>
    </div>
</div>

</div>
</div>
```

Bước 7: Khởi động ứng dụng

- Trở lại Visual Studio, bấm nút **Play** màu xanh lá cây ở thanh công cụ phía trên cùng (hoặc nhấn phím tắt F5 / Ctrl + F5).
- Chờ vài giây để Visual Studio biên dịch mã (Build). Nếu không có lỗi gì, trình duyệt mặc định của bạn (Chrome/Edge) sẽ tự động bật lên hiển thị trang chủ.

Bước 8: Kịch bản kiểm thử (test cases)

Bây giờ bạn hãy đóng vai một khách hàng và thực hiện các bước sau để xem Session hoạt động:

1. **Test Thêm sản phẩm:** Tại trang chủ, bấm nút "**Thêm vào giỏ**" dưới đôi *Giày chạy bộ*. Bạn sẽ ngay lập tức được giữ nguyên ở trang chủ (nhờ logic returnUrl thông minh), nhưng sản phẩm đã được lưu ngầm vào Session.
2. **Test Xem giỏ hàng:** Gõ thêm `/Cart` vào thanh URL trình duyệt (ví dụ: `localhost:xxxx/Cart`). Giao diện giỏ hàng hiện ra, hiển thị 1 đôi giày giá \$100.
3. **Test Cộng dồn:** Bấm nút "**Tiếp tục mua sắm**", hệ thống đưa bạn về lại trang chủ. Bạn hãy bấm thêm *Giày chạy bộ* một lần nữa, sau đó quay lại `/Cart`. Lúc này, số lượng tự động cập nhật thành 2, và thành tiền là **\$200**.
4. **Test Đa dạng sản phẩm:** Ra trang chủ thêm *Vợt cầu lông*. Vào giỏ hàng kiểm tra, bạn sẽ thấy 2 dòng sản phẩm riêng biệt và phần **Tổng cộng** tính đúng thành **\$250**.
5. **Test Xóa sản phẩm:** Bấm nút "**Xóa**" màu đỏ ở dòng *Vợt cầu lông*. Sản phẩm bay màu ngay lập tức, giỏ hàng tự động tải lại và tiền tổng giảm xuống còn \$200.

CÂU HỎI LÝ THUYẾT

1. Tại sao giao thức HTTP được gọi là "stateless"? Điều này gây ra khó khăn gì khi xây dựng một trang web thương mại điện tử?
2. Hãy mô tả luồng hoạt động của Session bằng ví dụ "phòng gửi đồ" bạn đã học. Session ID được lưu trữ ở đâu? Dữ liệu của giỏ hàng được lưu trữ ở đâu?
3. Trong file `Program.cs`, ba dòng code nào là cần thiết để kích hoạt Session? Nêu vai trò của từng dòng.
4. Extension method là gì? Trong cú pháp `public static void MyMethod(this string s)`, từ khóa `this` có ý nghĩa gì?

TỔNG KẾT BÀI HỌC

Tuần này, chúng ta đã giải quyết một trong những thách thức của lập trình web:

- Chúng ta đã hiểu rõ bản chất "**hay quên**" của **HTTP** và các cách để khắc phục.
- Chúng ta đã làm chủ được **Session**, một công cụ mạnh mẽ và an toàn để duy trì trạng thái phía server.

- Chúng ta đã học được một kỹ thuật lập trình nâng cao là **Extension Methods** để làm cho code của mình gọn gàng và dễ tái sử dụng hơn.

Với những kiến thức này, bạn đã sẵn sàng để xây dựng các tính năng tương tác phức tạp, bắt đầu với chức năng giỏ hàng ở bài thực hành.

BÀI TẬP VỀ NHÀ

1. Thực hiện đầy đủ các bước cấu hình Session trong file Program.cs của dự án SportsStore.
2. Tạo file SessionExtensions.cs và viết lại hai phương thức SetJson và GetJson theo hướng dẫn.
3. Trong HomeController, thử nghiệm lưu và đọc một chuỗi đơn giản vào Session để kiểm tra xem cấu hình của bạn đã hoạt động đúng chưa.

```
public IActionResult Index() {  
    HttpContext.Session.SetString("Test", "Hello Session!");  
    ViewBag.SessionTest = HttpContext.Session.GetString("Test");  
    return View(_repository.Products);  
}
```

Sau đó, trong Index.cshtml, hiển thị @ViewBag.SessionTest để xem kết quả.

TÀI LIỆU THAM KHẢO

- [1]. Session and state management in ASP.NET Core: Microsoft Docs - Session
- [2]. Extension Methods (C# Programming Guide): Microsoft Docs - Extension Methods
- [3]. JSON serialization and deserialization in .NET: Microsoft Docs - System.Text.Json